Weather Intergrated Crop Health Monitoring: A Modular ResNet-18 + Rule-Based Risk Layer

Khang Phan
EGN6217 – Applied Deep Learning
Spring 2026
github.com/itsmekhang/crop-health-monitoring

***Abstract*—This paper documents the first full implementation of a modular crop health system that combines an image-based disease classifier with a weather-informed risk layer. The model PlantVillage classifies leaf disease across 39 classes covering Tomato, Potato, and Pepper based on the Resnet-18 architecture. An XGBoost model then uses a rule-based risk formula using the disease output and live weather conditions from the Open-Meteo API. Both models are integrated into a Streamlit dashboard that provides real-time feedback, environmental risk assessment, and estimated revenue loss. This report outlines the implementation executions, an honest evaluation of both models, and the main challenges faced during this particular phase, specifically class imbalance, the lab-to-field generalization gap, and the inherent nature of the XGBoost training target.**

I. Project Summary

A. Purpose

Farmers still rely on slow and tedious manual disease scouting or delayed laboratory testing, leading to late mitigation and unnecessary crop losses. This project builds a modular AI prototype that processes a leaf photograph of a crop plant and a location, and returns a disease diagnosis, an environmental risk score, a treatment plan, and an estimated revenue impact. The goal is a functioning, realistic prototype.

B. Progress Since Deliverable 1

Deliverable 1 established the project scope, planned architecture, and data sources. Since then, the following has been completed:

- ResNet-18 disease classifier trained and saved to `results/disease_model.pth`
- XGBoost environmental risk model trained and saved to `results/risk_model.pkl`
- Streamlit interface (`ui/app.py`) integrating both models with live weather
- Accuracy evaluation per-class and confusion matrix visualization
- Dataset scope confirmed: 39 disease/healthy classes across 14 crops; class imbalance recorded
- XGBoost framing revised: now explicitly described as a rule approximator with a derived target

C. The Remaining

The following improvements are planned before the next deliverable: running the full per-class accuracy table against the trained model and record those values; testing the prototype with real field photos to evaluate the lab-to-field gap; and exploring whether fine-tuning deeper ResNet layers can improve the accuracy on under-represented classes.

II. System Architecture and Pipeline

Fig. 1 shows the full data flow. The two models operate independently and are combined only at the output stage. This is a deliberate design choice: it avoids the intergration complexity of true multimodal fusion and keeps each component independable.

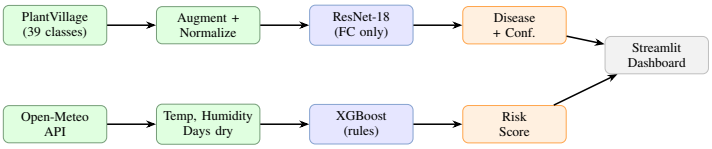

Fig. 1. System pipeline. ResNet-18 processes the leaf image independently of weather; XGBoost combines the disease output with environmental features.

A. Model Roles

ResNet-18 is the academic center of the project. It performs the main disease classification task: given a leaf image, predict which of the 15 disease or healthy classes it belongs to. Weather does not enter this model, a diseased leaf looks the same regardless of humidity.

XGBoost is a decision-support layer. It takes the predicted disease class, the classifier confidence, and three weather features (temperature, humidity, days since rain) and outputs a risk score in $[0, 1]$. This score dictates the vulnerability level and treatment recommendation displayed in the UI. The training target is derived from logic-based rules in `src/fusion.py`, not from real-world observations (see Section V).

III. Model Implementation Details

A. Disease Classifier – ResNet-18

Framework: PyTorch with torchvision.

Architecture: ResNet-18 initialized with ImageNet weights. All convolutional layers are frozen; only the final fully connected layer is fine-tuned. The replacement head maps from 512 features to $C = 15$ classes.

Training setup:

- Hardware: T4 GPU with high RAM enabled

- Epochs: 10
- Batch size: 32
- Optimizer: Adam, lr $= 10^{-3}$
- Scheduler: StepLR, step_size $= 3$, $\gamma = 0.5$
- Loss: CrossEntropyLoss

Dataset split: Two separate `ImageFolder` datasets are constructed from `data/raw/PlantVillage` – one with training augmentations, one with only normalization. A `Subset` object with a fixed random permutation splits 80% to training and 20% to validation. Using two separate datasets avoids the transform errors and problems that occurs when a single `ImageFolder` object has its `.transform` reassigned mid-training.

Augmentations: Random horizontal flip, rotation $\pm 15°$, color jitter (brightness/contrast ± 0.2), then ImageNet normalization.

Reproducibility: The training notebook runs end-to-end from dataset download to model save.

B. Environmental Risk Layer – XGBoost

Framework: XGBoost with scikit-learn API.

Architecture: XGBRegressor with 300 estimators, max_depth $= 5$, learning rate $= 0.05$, subsample $= 0.8$, colsample_bytree $= 0.8$.

Features: temperature (°C), humidity (%), days since rain, confidence, label-encoded disease class.

Target construction: The training set is generated synthetically. For each disease class, 500 (disease, weather) samples are drawn from disease-specific weather distributions (e.g., fungal diseases are sampled with high humidity). The risk score label is then computed by the deterministic formula in `src/fusion.py`:

$$s = 0.4 \cdot b_{\text{class}} + 0.3 \cdot c + 0.3 \cdot f_{\text{env}}(t, h, r) \tag{1}$$

where b_{class} is a fixed per-class severity base, c is classifier confidence, and f_{env} is a weighted environmental score.

This derivation is the key answer for interpreting XGBoost performance (Section V).

C. Interface

The Streamlit dashboard (`ui/app.py`) takes one or more leaf images, a location text (via the Open-Meteo API), and economic inputs (field size in acres, crop price per lb). For each image, it executes the disease classifier, queries current weather for the entered location, passes both through the XGBoost risk layer, and displays: disease class and top-5 predictions with confidence metrics, risk level and score, treatment plan and prevention notes, and estimated revenue loss based on USDA NASS yield database.

IV. Interface Prototype

A. Inputs and Outputs

TABLE I
INTERFACE INPUT/OUTPUT SUMMARY

Panel	Input	Output
Leaf Images	1+ image files	Disease class, confidence
Weather	City / region	Temp, humidity, days dry
Economic	Acres, price/lb	Revenue loss estimate
Combined per image		Risk level + treatment plan

B. Limitations

The interface does not perform input validation on image quality (blurry or non-leaf photos will silently produce a confident but meaningless prediction). If no real location is entered, the application will go with default values. Revenue loss estimates are based on the US national average yields and are not specifically oriented to the user's region.

C. Screenshots

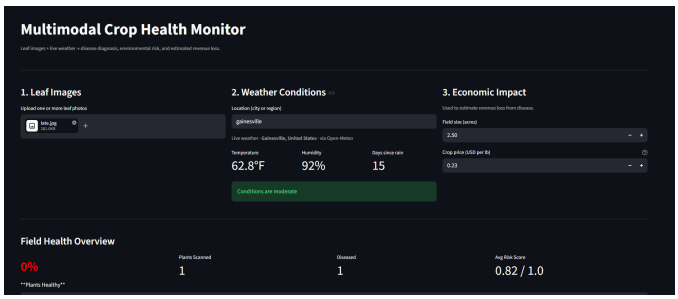

Fig. 2. Input panel: image upload, live weather fetch, and economic inputs.

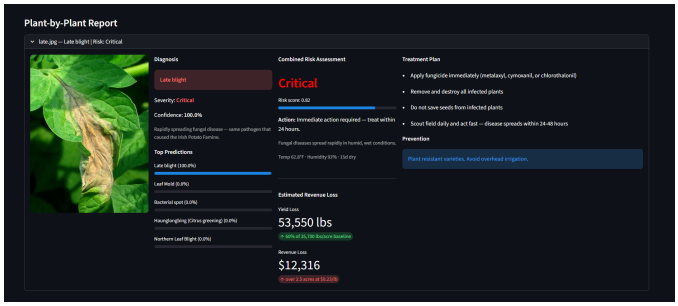

Fig. 3. Per-plant report: disease diagnosis, risk score, treatment plan, and revenue loss estimate.

V. Early Evaluation and Results

A. Dataset Characteristics

TABLE II
PlantVillage Crop Coverage (39-class full dataset)

Crop	Classes	Disease / Healthy
Tomato	10	9 disease, 1 healthy
Apple	4	3 disease, 1 healthy
Grape	4	3 disease, 1 healthy
Corn	4	3 disease, 1 healthy
Potato	3	2 disease, 1 healthy
Pepper	2	1 disease, 1 healthy
Cherry	2	1 disease, 1 healthy
Peach	2	1 disease, 1 healthy
Strawberry	2	1 disease, 1 healthy
Others	6	1 disease, 5 healthy
Total	**39**	**26 disease, 13 healthy**

The full PlantVillage dataset covers 14 crop types across 39 classes (≈54,000 images). Class imbalance remains significant: Tomato accounts for the majority of disease classes and images. Validation accuracy overestimates practical performance on minority classes such as Blueberry, Raspberry, and Soybean, which have only healthy examples.

B. ResNet-18 Training Curves

Training is recorded per epoch. Fig. 4 and Fig. 5 show the per-epoch loss and accuracy for training and validation splits. Fine-tuning deeper layers is identified as a future improvement.

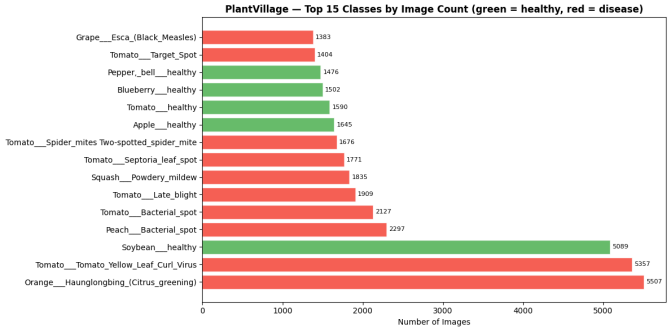

Fig. 4. ResNet-18 training and validation loss per epoch. The model converges within 3–5 epochs under frozen-backbone transfer learning.

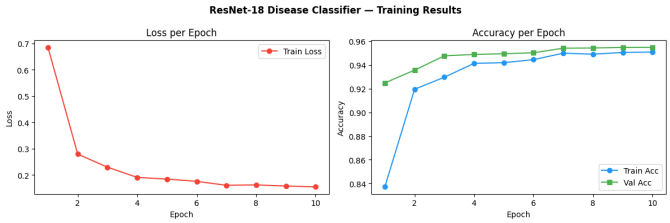

Fig. 5. ResNet-18 training and validation accuracy per epoch. Validation accuracy plateaus as the frozen backbone limits representational capacity.

C. Per-Class Accuracy

Per-class accuracy (calculated in `setup.ipynb`) is the primary evaluation metric. Fig. 6 shows the per-class breakdown; majority Tomato classes (large sample sizes) achieve high accuracy while minority classes – specifically *Potato_healthy* and *Tomato_mosaic_virus*, show lower and more ambiguous accuracy. Any top-line validation accuracy figure should be reported with this breakdown to avoid misrepresenting the model quality. Fig. 7 shows the confusion matrix; preliminary analysis suggests visually similar fungal diseases (Early vs. Late blight, Bacterial spot vs. Septoria) produce the most off-diagonal confusion.

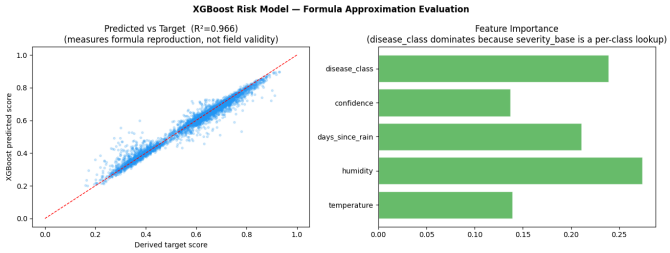

Fig. 6. Per-class validation accuracy across all 39 PlantVillage classes. Minority classes (low sample counts) show noticeably higher variance.

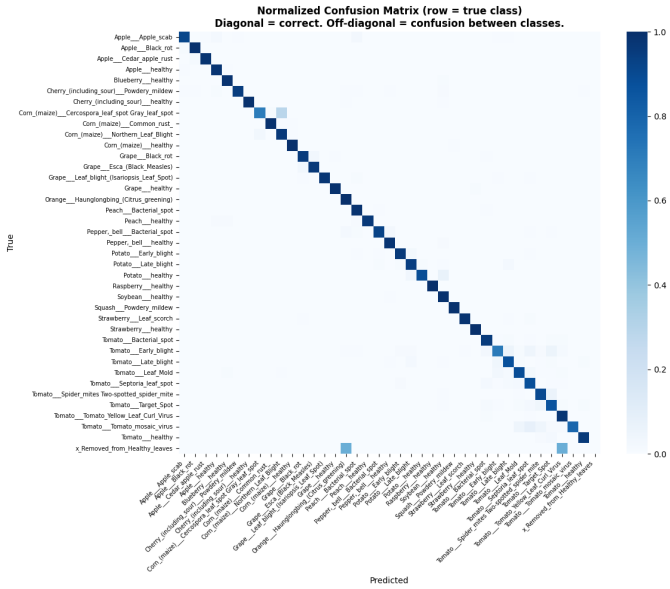

Fig. 7. Confusion matrix on the validation split. Off-diagonal clusters correspond to visually similar disease pairs (e.g., Early vs. Late blight).

D. XGBoost Risk Model – Honest Interpretation

The XGBoost model achieves Val MAE ≈ 0.0184 and Val R^2 ≈ 0.9661. These numbers look fantastic but **do not measure real-world validity**. The training target is the deterministic output of Equation 1 – a formula defined entirely in code. XGBoost is learning to compute that formula, not

to interpret from field observations. This is confirmed by the feature vs risk score shown in Fig. 8.

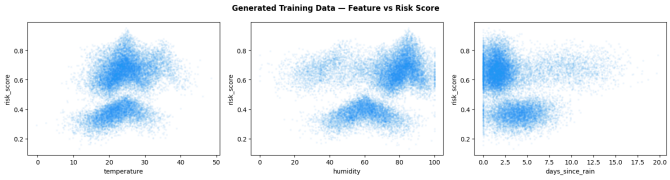

Fig. 8. XGBoost Feature vs Risk Score

The practical value of the XGBoost layer is that it packages the rule into a compact, interpretable scorer that handles constant weather inputs without explicit branching. It should be described as a rule interpreter and evaluated on whether the pre-determined hard-coded rules are sensible, not on R^2 alone.

VI. Challenges and Next Steps

A. Technical Challenges

Data Distribution The 21× ratio between the most and least frequent class degrades recall on minority classes. Options for the next deliverable include weighted sampling during training or class-weighted loss.

Real-world samples. All PlantVillage images are photographed using uniform backgrounds under controlled lighting. Early informal tests with field photos show noticeably lower confidence. This discrepancy must be accounted for and reported.

Random split inflation. PlantVillage contains near-duplicate images of the same leaf taken seconds apart. A random 80/20 split places near-duplicates in both train and validation sets, inflating reported accuracy. A plant-session or leaf-level split would provide a more reserved prediction.

XGBoost derived target. As discussed in Section V-D, the risk model's R^2 reflects formula reproduction, not predictive validity. The report and UI must be clear about this.

B. Next Steps

- Test the interface on real field photos; report the qualitative accuracy drop
- Explore weighted loss or oversampling for minority classes (Potato_healthy)
- Consider fine-tuning the last ResNet block rather than only the FC layer
- In the final report, reframe the XGBoost section clearly as a rule-based decision layer with a transparent derivation of the risk formula assumptions

VII. Responsible AI Reflection

Lab-to-field bias. The most significant concern is geographic and environmental bias. PlantVillage images come from controlled laboratory settings. Farmers in regions with different soil backgrounds or different lighting conditions may yield worst performance. This problem is not dedicated for the discussion section; it must appear in how classifier results are interpreted and in any accuracy claim made about the system.

Class imbalance and representation. The dataset over-represents Tomato diseases (77.5% of images). A system deployed to Potato or Pepper farmers will be less reliable than headline accuracy suggests. Per-class reporting is the minimum; separate deployment testing per crop is the realistic standard.

Transparency and over-reliance. The user interface outputs top-5 predictions and a confidence score to discourage binary trust. The XGBoost risk score is derived from logic-based coded rules, not from reported field observations, and this is now stated explicitly in the interface's header and in the training notebook. A farmer acting on a Critical risk alert should understand that the score is the output of an set formula applied to a CNN prediction, not an empirically validated risk estimate.

Misuse. An incorrect Critical diagnosis could lead a farmer to take unnecessary mitigation actions, incurring cost and environmental burden. The UI is designed to recommend professional verification for Critical and High risk levels. No automated action is taken; all outputs are advisory.

Computational footprint. ResNet-18 is a lightweight architecture. Training for 10 epochs on 41k images takes approximately 5–10 minutes on a single GPU. The XGBoost model trains in under one minute on CPU. The inference footprint is minimal and appropriate for deployment on low-cost hardware.

GitHub: itsmekhang/crop-health-monitoring